

UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

Cómputo Móvil · Grupo 03 · Semestre 2026-2



Trabajo de investigación y análisis técnico

AdaptaMente

Aplicación móvil iOS para educación adaptativa con inteligencia artificial centrada en el humano

Equipo HydraSwift

Caballero Montaña Juan Manuel

Camarena Arévalo Yael Eduardo

Cruz Soria Oscar

Profesor: Ing. Marduk Pérez de Lara Domínguez

Mayo de 2026

Tabla de contenido

Tabla de contenido	2
Resumen ejecutivo	3
1. Análisis de requerimientos	3
1.1 Reglas de negocio	3
1.2 Requerimientos funcionales	4
1.3 Requerimientos no funcionales	4
2. Viabilidad técnica	4
2.1 Funcionalidades viables que sí implementamos	5
2.2 Funcionalidades que descartamos por inviables	5
3. Alcance y MVP	5
3.1 Qué entra al MVP	6
3.2 Qué queda fuera del MVP	6
3.3 Criterios de aceptación del MVP	6
4. Wireframes y documentación por pantalla	6
4.1 Splash y Welcome (pantallas 01-02)	6
4.2 Cuestionario VAK y selección de rol (pantallas 03-04)	7
4.3 Generador docente y hub del estudiante (pantallas 05-06)	8
4.4 Adaptaciones visual y auditiva (pantallas 07-08)	9
4.5 Adaptación kinestésica y hub de modos accesibles (pantallas 09-10)	10
4.6 Modos Dislexia y Foco TDAH (pantallas 11-12)	11
4.7 Unirme a una clase (pantalla 13)	12
5. Flujo de navegación y cambios de estado	13
5.1 Diagrama de estados resumido	14
6. Gestos e interacciones especiales	14
7. Datos, servicios y almacenamiento	15
7.1 Servicio externo: Google Gemini API	15
7.2 Servicios nativos del sistema operativo	15
7.3 Operaciones por pantalla y almacenamiento local	16
8. Dispositivos, tamaños, orientaciones y sensores	16
8.1 Dispositivos soportados	16
8.2 Sensores y capacidades de hardware	17
9. Maqueta de pantallas finales	17
9.1 Apertura y menú personalizado	17
9.2 Generación del docente y versión visual	18
9.3 Publicación de aula y modo Dislexia	18
10. Lenguajes y herramientas de desarrollo	19
10.1 Herramientas de desarrollo	19
11. Arquitectura y patrones de diseño	20
11.1 Capas de la arquitectura	20
11.2 Patrones de diseño aplicados	20
12. Metodología de desarrollo y administración del proyecto	20
12.1 Herramientas de administración	20
12.2 Sprints y entregas internas	21
13. Uso de IA durante nuestro desarrollo	21
14. Permisos y consideraciones para publicación en App Store	21
14.1 Programa Apple Developer	21
14.2 App Store Review Guidelines relevantes	22
14.3 Privacy Manifest y App Store Connect	22
15. Equipo de trabajo y roles	22
16. Estimaciones de tiempo y costo	22
16.1 Tiempo invertido en el MVP (real)	22
16.2 Estimación proyectada a producción comercial	23
17. Referencias	23

Resumen ejecutivo

AdaptaMente es una aplicación nativa para iOS que nuestro equipo desarrolló en el Hackathon 2026 por parte del curso de Cómputo Móvil y que aquí analizamos a fondo desde el punto de vista técnico. La idea de fondo es sencilla. Toma el material que un docente ya tiene preparado y lo reescribe, con ayuda de IA generativa, en tres formatos sensoriales que conviven al mismo tiempo (visual, auditivo y kinestésico). A eso le sumamos dos modos pensados para neurodiversidad, uno para dislexia y otro para TDAH, que se apartan a propósito del modelo VAK clásico porque cada perfil cognitivo tiene problemas concretos que el VAK no resuelve.

El hilo conductor del trabajo es la temática del curso, la IA centrada en el humano (Shneiderman, 2020). Cada decisión técnica que aparece en las páginas siguientes la justificamos apoyándonos en dos ideas que volvimos operativas. La primera es que el docente conserva el control, porque nada de lo que produce la IA llega al estudiante sin que él lo haya revisado y aprobado. La segunda es que el perfil cognitivo del alumno se queda en su teléfono y no viaja a ningún servidor.

El documento respeta la estructura que pidió el profesor. Recorremos, en este orden, los requerimientos de negocio y técnicos, la viabilidad de lo que propusimos construir, el alcance del MVP, los wireframes de las trece pantallas con su ficha de datos, el análisis de flujo y de cambios de estado, los servicios externos y el almacenamiento local, los dispositivos y sensores, las maquetas finales con datos reales, la pila tecnológica, la arquitectura, la administración del proyecto, cómo usamos IA en nuestro propio proceso, los frameworks empleados, lo necesario para publicar en la App Store, la conformación del equipo con sus roles y, al final, las estimaciones de tiempo y costo. Cierra con las referencias.

1. Análisis de requerimientos

Esta sección abre los tres niveles de requerimientos que ordenan la aplicación. Arrancamos por las reglas de negocio, que son los compromisos que no estamos dispuestos a romper. Seguimos con los requerimientos funcionales, que dicen qué tiene que hacer la app, y terminamos con los no funcionales, que fijan cómo debe hacerlo. Cada bloque lo amarramos a la temática de IA centrada en el humano.

1.1 Reglas de negocio

Las reglas de negocio nacen de nuestra apuesta por la IA centrada en el humano y funcionan como el contrato del producto. Toda funcionalidad nueva tiene que respetarlas y, si alguna las incumple, no sale a producción. Estas son:

Código	Regla de negocio	Justificación (IA centrada en el humano)
RN-01	La app trabaja con dos roles separados, maestro y estudiante. Cualquier persona puede tomar uno u otro sin crear una cuenta.	El maestro conserva el control pedagógico y el estudiante recibe material ya curado.
RN-02	Todo lo que genera la IA debe pasar por una revisión y una aprobación explícita del docente antes de publicarse.	Lleva a la práctica la idea de Shneiderman de combinar alta automatización con alto control humano.
RN-03	El perfil cognitivo del estudiante (lo que arroja el cuestionario VAK) vive solo en el dispositivo y no se sincroniza a la nube.	Privacidad por diseño del dato sensible. Además se alinea con la minimización que pide el Privacy Framework de Apple.
RN-04	El modelo VAK se le muestra al usuario como una guía de trabajo, nunca como una etiqueta que define lo que puede o no puede aprender.	Asume la crítica de Pashler et al. (2008) y mueve el foco hacia ofrecer alternativas en vez de imponer un solo formato.
RN-05	Los modos de neurodiversidad (dislexia y TDAH) son opcionales y jamás clasifican médicamente a nadie.	Evita estigmatizar, ya que son apoyos, no diagnósticos.
RN-06	La app sigue siendo útil sin internet gracias a un motor local que degrada el servicio con elegancia.	Asegura acceso parejo donde la conectividad es limitada, algo muy real en el contexto mexicano.
RN-07	El modelo de negocio es freemium en tres capas y descartamos la publicidad como forma de monetizar.	La publicidad mete incentivos que chocan con el bienestar pedagógico, sobre todo tratándose de menores.

1.2 Requerimientos funcionales

Los requerimientos funcionales describen lo que la aplicación tiene que ser capaz de hacer. Los agrupamos por flujo de usuario para que sea fácil seguirles el rastro hasta las pantallas que documentamos en el capítulo 4. Todos están implementados en el código que entregamos como MVP.

Código	Requerimiento funcional	Pantalla principal
RF-01	Mostrar una pantalla de carga inicial con animación durante dos segundos.	SplashView
RF-02	Permitir que el usuario elija su rol al entrar (maestro o estudiante).	WelcomeView
RF-03	Pedir al estudiante que conteste diez preguntas para detectar su estilo dominante.	LearningStyleTestView
RF-04	Dejar que el docente cargue contenido de dos formas: importando un PDF o pegando texto plano.	TeacherGeneratorView
RF-05	Generar tres adaptaciones del mismo contenido (visual, auditiva y kinestésica) en menos de ocho segundos.	TeacherGeneratorView
RF-06	Permitir que el docente edite a mano cada adaptación antes de aprobarla.	TeacherGeneratorView
RF-07	Publicar el material aprobado bajo un código único de aula.	TeacherGeneratorView
RF-08	Permitir que el estudiante entre a un aula publicada escribiendo su código.	JoinClassView
RF-09	Permitir que el estudiante estudie material propio en modo autodidacta, sin necesidad de un aula.	SelfStudyView
RF-10	Reproducir el contenido auditivo con voz nativa en español mexicano y control de velocidad.	AudioAdaptationView
RF-11	Presentar el contenido kinestésico como tarjetas interactivas en tres modalidades (flashcards, emparejamiento y secuencia).	KinestheticAdaptationView
RF-12	Ofrecer un modo Dislexia con cuatro ajustes independientes (fuente, fondo crema, espaciado y resaltado sincronizado).	DyslexiaModeView
RF-13	Ofrecer un modo Foco TDAH con fragmentación intensa, deslizamiento con háptica y una pausa de respiración cada cinco tarjetas.	TDAHFocusModeView
RF-14	Resaltar la palabra activa en sincronía con el sintetizador durante la lectura en voz alta.	DyslexiaModeView
RF-15	Seguir operando con un motor local de NLP cuando la API de Gemini no esté disponible.	Servicio transversal

1.3 Requerimientos no funcionales

Los requerimientos no funcionales marcan los umbrales de calidad. A diferencia de los funcionales, no dicen qué hace el sistema, sino bajo qué condiciones consideramos aceptable que lo haga.

Categoría	Métrica	Umbral
Rendimiento	Generación completa de las tres adaptaciones VAK	≤ 8 segundos con la API de Gemini
Rendimiento	Arranque en frío hasta SplashView	≤ 1.5 segundos en iPhone 12 o superior
Privacidad	Datos del estudiante enviados a servidores externos	Cero (apego estricto al Privacy Framework)
Disponibilidad	Operación sin conexión con el motor local	100 % de las funciones, salvo la generación de infografías premium
Compatibilidad	Versión mínima del sistema operativo	iOS 17.0
Compatibilidad	Modelos soportados	iPhone SE (3.ª gen) hasta iPhone 16 Pro Max
Accesibilidad	Compatibilidad con Dynamic Type, VoiceOver y Reduce Motion	Total, en todas las pantallas
Localización	Idiomas del MVP	Español neutro y español mexicano (TTS)
Usabilidad	Profundidad máxima de navegación desde el menú principal	Tres niveles
Seguridad	Datos sensibles persistidos con cifrado adicional al de iOS	Ninguno (no guardamos datos sensibles)
Estabilidad	Tasa máxima de fallo en sesión sin red	≤ 1 %

2. Viabilidad técnica

Durante la planeación nos sentamos a revisar una por una las funcionalidades que habíamos propuesto, para ver cuáles cabían en el plazo del hackathon y en los recursos del equipo. Miramos tres cosas en cada caso: qué tan complicada era técnicamente, cuánto costaba y cuánto tiempo nos llevaría. Abajo contamos las decisiones que más pesaron, tanto las funciones que conservamos como las que dejamos fuera.

2.1 Funcionalidades viables que sí implementamos

Generar tres adaptaciones VAK al mismo tiempo con Gemini 2.5 Flash salió viable porque el SDK responde en menos de ocho segundos para entradas de hasta cuatro mil tokens, y porque el costo por petición es prácticamente nulo en los volúmenes que manejamos durante las pruebas y las demostraciones. Toda la integración cupo en un solo servicio, AIAdaptationService, que convierte la salida del modelo a nuestras estructuras Codable.

Procesar el cuestionario VAK en el propio dispositivo fue casi trivial: una clasificación por conteo ponderado y normalización softmax cabe en menos de doscientas líneas y corre en milisegundos. Quedó en el servicio LearningStyleClassifier.

La síntesis de voz en español mexicano también fue viable porque AVSpeechSynthesizer ya trae voces nativas de buena calidad y sin costo. Lo difícil ahí fue la sincronización palabra por palabra del modo Dislexia, que resolvimos suscribiéndonos al método delegado willSpeakRangeOfString del sintetizador y publicando el rango activo a las vistas con un NSRange marcado como @Published.

El modo Foco TDAH, pese a ser lo más novedoso, salió adelante reutilizando piezas que iOS ya ofrece sin costo: DragGesture, animaciones spring y la háptica de UIImpactFeedbackGenerator. La pausa de respiración la armamos como un overlay con un Timer que rota tres fases de cuatro segundos cada una.

2.2 Funcionalidades que descartamos por inviables

De las propuestas iniciales del equipo, cuatro quedaron fuera porque no cabían en el alcance del MVP. Las dejamos documentadas porque forman parte del trabajo de ingeniería y porque ayudan a entender hasta dónde decidimos llegar a conciencia.

La primera fue leer pizarrones físicos con la cámara mediante reconocimiento óptico. iOS tiene VisionKit y reconoce texto con buena precisión, pero el caso real obliga a corregir la perspectiva, separar columnas, distinguir diagramas dibujados a mano y lidiar con tinta de bajo contraste. Calculamos que dejarlo con calidad aceptable nos habría costado al menos dos semanas más, y preferimos invertir ese tiempo en pulir las adaptaciones VAK y los modos de neurodiversidad.

La segunda fue entrenar un modelo propio. Apple permite incrustar modelos Core ML personalizados, pero entrenar uno de generación de texto adaptativo nos habría exigido un dataset etiquetado que no tenemos y cómputo en la nube por encima del presupuesto. La dependencia de un modelo externo (Gemini) la cubrimos con una degradación elegante hacia un motor local basado en NLTokenizer y reglas heurísticas, que mantiene el servicio aunque baje la calidad.

La tercera fue sincronizar en tiempo real los dispositivos de un mismo profesor, por ejemplo generar en el iPhone y revisar en el iPad. CloudKit es gratis hasta cierto punto, pero suma complejidad de resolución de conflictos y, lo más importante, contradice nuestra regla RN-03 sobre privacidad. Optamos por dejar todo en almacenamiento local y compartir a través del código de aula.

La cuarta fue una versión web en paralelo con SwiftUI for Web o catalizadores. El SDK de SwiftUI para web todavía no está maduro y el equipo no domina stacks web en el periodo del hackathon. Lo registramos sin rodeos en nuestro FODA como una debilidad activa (solo iOS, cuando Android tiene el 78 % del mercado mexicano), pero la aceptamos como un costo razonable para el MVP.

3. Alcance y MVP

El producto mínimo viable que entregamos cubre el ciclo entero de generar, revisar, publicar y consumir material educativo adaptado, lo mismo en la modalidad de aula con código que en la autodidacta. Enseguida delimitamos qué entra y qué queda afuera, y dejamos apuntado lo que haríamos si el proyecto continuara después del curso.

3.1 Qué entra al MVP

El MVP incluye todo el onboarding (Splash, Welcome y el cuestionario VAK), la elección de rol, el generador completo del docente con sus tres adaptaciones simultáneas y su panel de aprobación y publicación, el hub del estudiante con sus tres entradas (clase con código, autodidacta y modos accesibles), las tres vistas de consumo VAK, los dos modos de neurodiversidad con todos sus toggles funcionando y, por debajo, los servicios transversales de TTS, clasificación, almacenamiento de aulas y orquestación de IA.

Metimos los dos modos de neurodiversidad al MVP a propósito, aun sabiendo que añaden complejidad, porque diferencian la propuesta frente a otras apps y porque hacen tangible la idea de la IA centrada en el humano: el contenido se adapta a la persona y no al revés.

3.2 Qué queda fuera del MVP

Cinco bloques quedaron fuera. El primero es la sincronización en la nube entre dispositivos de un mismo profesor, que como ya dijimos choca con RN-03. El segundo es la versión Android nativa, que obligaría a reescribir la capa de presentación en Jetpack Compose. El tercero es la analítica docente avanzada (cuántos alumnos terminaron qué adaptación, en cuánto tiempo y con qué retención por estilo), que dejaríamos para una versión futura más completa. El cuarto es la integración con plataformas de gestión escolar como Google Classroom o Microsoft Teams for Education, que requiere acuerdos comerciales y endpoints OAuth fuera del plazo del curso. El quinto es el contenido multimedia rico (videos animados, esquemas 3D interactivos). Es posible con Apple Vision Pro y RealityKit, pero se sale del alcance del MVP para iPhone.

3.3 Criterios de aceptación del MVP

Damos el MVP por terminado cuando se cumplen cinco criterios medibles. El primero, que un docente sin experiencia previa pueda generar y publicar material adaptado en menos de cinco minutos desde que abre la app por primera vez. El segundo, que un estudiante pueda entrar a un aula y consumir el material en su estilo preferido en menos de un minuto. El tercero, que ambos modos de neurodiversidad funcionen sin internet. El cuarto, que la app no falle durante una demostración típica de tres minutos. Y el quinto, que se pueda verificar, inspeccionando el tráfico de red, que el perfil cognitivo del estudiante se procesa de forma local. Validamos los cinco en pruebas internas con un iPhone 13 y un iPhone 15 Pro.

4. Wireframes y documentación por pantalla

Presentamos los wireframes de baja fidelidad de las trece pantallas principales. Para cada una incluimos una ficha compacta con la información que muestra, los tipos de dato que maneja, su vigencia, de dónde viene el dato y qué operaciones esperamos sobre él. Los wireframes se trazaron a tamaño iPhone (393 × 852 puntos) y reflejan la geometría de las vistas que ya están implementadas en Swift. La versión visual final, con colores y datos reales, se muestra en el capítulo 9.

4.1 Splash y Welcome (pantallas 01-02)

9:41

5G



Figura 4.1. Wireframe de SplashView

9:41

5G

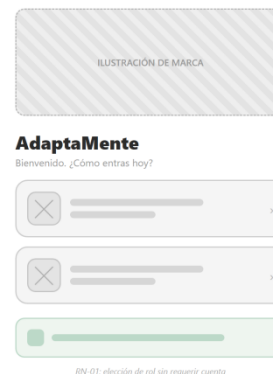


Figura 4.2. Wireframe de WelcomeView

SplashView (pantalla 01) es lo primero que ve el usuario al abrir la app. Se queda dos segundos con animaciones discretas de aparición y un fundido de medio segundo antes de ceder el paso al router principal. Está hecha en SwiftUI puro con Shapes nativos, sin un solo PNG. WelcomeView (pantalla 02) presenta la marca y plantea la primera decisión, elegir entre maestro o estudiante. Si elige maestro, se salta el cuestionario VAK, y si elige estudiante, pasa al diagnóstico. Aquí se aplica la regla RN-01.

Atributo	Detalle
Información mostrada	Splash: logo Brain-A, marca y tagline. Welcome: ilustración de marca, mensaje de bienvenida, dos botones de rol y banner de privacidad.
Tipo de dato	Splash: estado de animación local. Welcome: UserProfile (parcialmente vacío) y un Bool hasCompletedOnboarding=false.
Vigencia	Splash: volátil. Welcome: la decisión queda guardada para siempre al elegir.
Origen	Splash: valores fijos en código. Welcome: UserStore observado vía @EnvironmentObject.
Operaciones esperadas	Splash: cambia el flag showSplash de la app al terminar. Welcome: modifica UserProfile.role y navega según el rol.

4.2 Cuestionario VAK y selección de rol (pantallas 03-04)



Figura 4.3. Wireframe de LearningStyleTestView

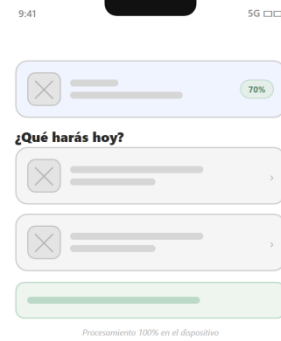


Figura 4.4. Wireframe de RoleSelectionView

El cuestionario (pantalla 03) lanza diez reactivos con tres opciones cada uno, una por estilo sensorial. La barra superior muestra el avance y, al terminar, se llama a `LearningStyleClassifier.shared` para calcular la distribución softmax y el estilo dominante. `RoleSelectionView` (pantalla 04) es el centro de mando tras el onboarding: saluda con el estilo identificado y ofrece dos accesos, maestro y estudiante.

Atributo	Detalle
Información mostrada	Quiz: pregunta actual, tres opciones, barra de progreso y contador. Role: nombre del usuario, estilo dominante, confianza y dos tarjetas de rol.
Tipo de dato	Quiz: un arreglo de <code>QuizQuestion</code> (Codable) y un diccionario <code>[LearningStyle: Double]</code> . Role: <code>UserProfile</code> completo y el enum <code>LearningStyle</code> .
Vigencia	Quiz: volátil, pero su resultado persiste. Role: persistente desde el primer uso.
Origen	Quiz: banco de preguntas embebido como JSON en el bundle. Role: el singleton <code>UserStore</code> (<code>UserDefaults</code>).
Operaciones esperadas	Quiz: lee el banco al <code>onAppear</code> y escribe el resultado. Role: solo lectura y navegación push.

4.3 Generador docente y hub del estudiante (pantallas 05-06)



Figura 4.5. Wireframe de TeacherGeneratorView

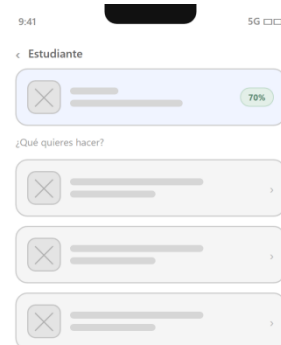


Figura 4.6. Wireframe de StudentHubView

TeacherGeneratorView (pantalla 05) es la pantalla con más funciones de la app: el docente carga un PDF o pega texto, le pone título, dispara la generación VAK, revisa el resultado, edita lo que quiera, aprueba y publica con un código de aula. Aquí vive la regla RN-02. StudentHubView (pantalla 06) es el menú del estudiante con tres entradas: unirse a un aula, estudiar por su cuenta y modos accesibles. Esta última la agregamos hacia el final del proyecto.

Atributo	Detalle
Información mostrada	Teacher: zona para soltar el PDF, campo de texto, título, tres tarjetas VAK, controles de edición y botón de aprobación. Student: estilo dominante, descripción y tres OptionCards.
Tipo de dato	Teacher: ImportedDocument, AdaptedContent (con VisualContent, AuditoryContent y KinestheticContent) y PublishedClass. Student: UserProfile (solo lectura).
Vigencia	Teacher: el AdaptedContent dura hasta que el maestro lo borra y la PublishedClass dura de forma indefinida. Student: persistente.
Origen	Teacher: el ImportedDocument llega vía PDFKit y el AdaptedContent vía Gemini 2.5 Flash. Student: UserStore.
Operaciones esperadas	Teacher: lee los prompts del bundle, hace un POST a Gemini y escribe en ClassroomStore al publicar. Student: lectura y navegación push.

4.4 Adaptaciones visual y auditiva (pantallas 07-08)



Figura 4.7. Wireframe de VisualAdaptationView

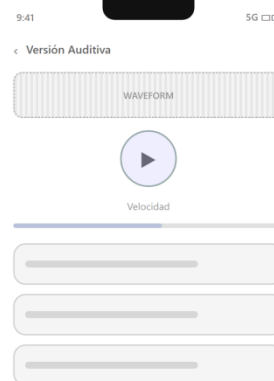


Figura 4.8. Wireframe de AudioAdaptationView

VisualAdaptationView (pantalla 07) arma el contenido para quien aprende mirando: un resumen en bullets, un mapa mental radial con nodo central y satélites que se pueden tocar, y una infografía con datos clave acompañados de iconos SF Symbols. AudioAdaptationView (pantalla 08) ofrece el contenido en audio: una waveform decorativa, el botón central de reproducción, un slider de velocidad y la lista de segmentos navegables. Por debajo se apoya en SpeechService, que maneja AVSpeechSynthesizer con voz nativa es-MX.

Atributo	Detalle
Información mostrada	Visual: resumen, mapa mental radial e infografía. Audio: waveform decorativa, botón de play, slider de velocidad y lista de segmentos.
Tipo de dato	Visual: VisualContent (summary, keyConcepts, mindMapNodes, Infographic). Audio: AuditoryContent (script, segments, estimatedDuration), un Float rate y un Bool isSpeaking.
Vigencia	Volátil dentro de la sesión. Solo persiste si el alumno entró por un aula publicada.
Origen	El AdaptedContent que llega del aula o del motor local NLP. Audio: delega en el singleton observable SpeechService.
Operaciones esperadas	Solo lectura del contenido. Audio: control sobre AVSpeech (speak, pause, stop, rate).

4.5 Adaptación kinestésica y hub de modos accesibles (pantallas 09-10)



Figura 4.9. Wireframe de KinestheticAdaptationView

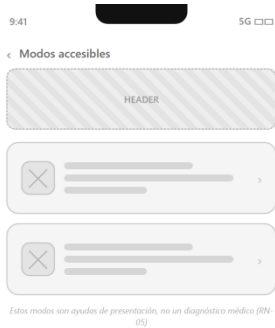


Figura 4.10. Wireframe de NeurodiverseHubView

KinestheticAdaptationView (pantalla 09) trae tres modalidades intercambiables: flashcards que se voltean al tocarlas, ejercicios de emparejar arrastrando y secuencias para ordenar pasos. Cada interacción dispara una háptica ligera. NeurodiverseHubView (pantalla 10) es la entrada a los modos pensados para neurodiversidad (Dislexia y TDAH), con un aviso al pie. Aplica la regla RN-05.

Atributo	Detalle
Información mostrada	Kinesthetic: un control segmentado de tres pestañas, la tarjeta interactiva activa y un contador. Neurodiverse: encabezado, dos tarjetas de modo (dislexia y TDAH) y un aviso.
Tipo de dato	Kinesthetic: KinestheticContent (flashcards, exercises, matchingPairs, orderingSteps). Neurodiverse: el enum NeurodiverseMode.
Vigencia	Kinesthetic: volátil. Neurodiverse: estado de interfaz volátil.
Origen	Kinesthetic: AdaptedContent. Neurodiverse: definido en código dentro de la propia vista.
Operaciones esperadas	Kinesthetic: lectura, seguimiento de progreso y háptica. Neurodiverse: solo navegación push al modo elegido.

4.6 Modos Dislexia y Foco TDAH (pantallas 11-12)

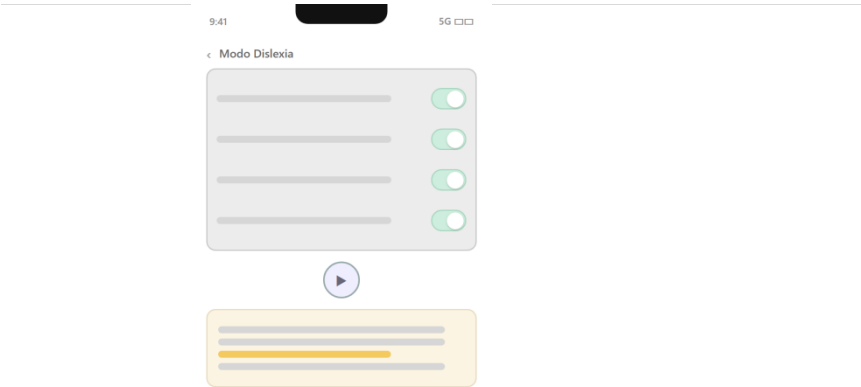


Figura 4.11. Wireframe de DyslexiaModeView

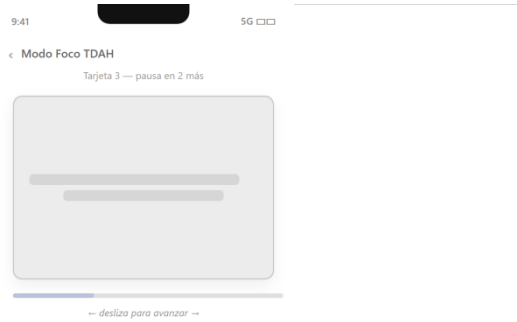


Figura 4.12. Wireframe de TDAHFocusModeView

DyslexiaModeView (pantalla 11) trae cuatro ajustes pensados para perfiles disléxicos: la fuente OpenDyslexic con respaldo en SF Pro Rounded Bold, fondo crema #FBF4E2, espaciado de letras +35 % mediante kern en NSAttributedString, y resaltado palabra por palabra sincronizado con AVSpeech. El texto se simplifica antes con TextAdaptationEngine.simplifyForDyslexia, siguiendo a Rello y Baeza-Yates (2013). TDAHFocusModeView (pantalla 12) lleva a la práctica el manejo de dopamina y atención sostenida que describe Sonuga-Barke (2003): el texto se parte en tarjetas de hasta 12 palabras, se muestra una sola a la vez sobre una pila 3D, el deslizamiento horizontal con umbral de 80 puntos dispara Haptics.medium, y cada cinco tarjetas la app se detiene con un overlay de respiración guiada de 30 segundos.

Atributo	Detalle
Información mostrada	Dyslexia: cuatro toggles, controles del reproductor y el texto simplificado con la palabra activa resaltada. TDAH: una sola tarjeta con una frase corta, contador, barra de progreso e indicador del gesto.
Tipo de dato	Dyslexia: los Bools de los toggles (NeurodiverseModeStore), un String con el texto y un NSRange con la palabra activa. TDAH: un Array<String> de tarjetas, un Int currentIndex, un Int cardsConsumedSinceBreak y un Bool showBreathBreak.
Vigencia	Los toggles y parámetros del modo persisten y el contenido adaptado es volátil.
Origen	NeurodiverseModeStore, SpeechService (delegate willSpeakRange) y el TextAdaptationEngine local.
Operaciones esperadas	Escribe en preferencias al cambiar un toggle, se suscribe de forma reactiva a las @Published y corre animaciones spring con háptica.

4.7 Unirme a una clase (pantalla 13)



Figura 4.13. Wireframe de JoinClassView

Esta pantalla deja que el estudiante entre a un aula publicada escribiendo el código alfanumérico de seis caracteres que aparece al pie del material aprobado. La búsqueda es local. ClassroomStore guarda un diccionario de PublishedClass indexado por código en UserDefaults. Si el aula existe, despliega una vista previa adaptada al estilo del alumno, y si no, muestra un error con animación de sacudida.

Atributo	Detalle
Información mostrada	Campo de código en tipografía monoespaciada, botón de conectar, lista de aulas disponibles en el dispositivo y tarjeta del material publicado.
Tipo de dato	Un String con el código, PublishedClass y AdaptedContent (por relación).
Vigencia	La PublishedClass persiste en el dispositivo del docente y en el de cada alumno conectado.
Origen	El singleton ClassroomStore.
Operaciones esperadas	Lectura sobre el diccionario indexado. Sin escrituras.

5. Flujo de navegación y cambios de estado

La app funciona como una máquina de estados anidada. En el nivel más alto está el router AdaptaMente1App, que alterna entre la pantalla de carga y el árbol completo de navegación según una variable @State llamada showSplash. Pasados los dos segundos del Splash, showSplash cambia a false y se monta ContentView, que a su vez decide si dibuja WelcomeView o RoleSelectionView según el valor de UserProfile.hasCompletedOnboarding.

Desde RoleSelectionView salen dos subflujos en paralelo. El del docente baja a TeacherGeneratorView y recorre los estados de ImportedDocument (cargado o vacío) y de AdaptedContent (generándose, listo para revisar y aprobado), y termina insertando una PublishedClass en el ClassroomStore. El del estudiante baja a StudentHubView y se ramifica en tres: JoinClassView con su búsqueda local por código, SelfStudyView con carga directa de PDF o texto, y NeurodiverseHubView con sus dos modos especializados.

Hay tres cambios de estado que pesan más que el resto. El primero es la mutación de UserProfile.hasCompletedOnboarding al terminar el cuestionario VAK, que manda al usuario de forma permanente al menú principal y se guarda en UserDefaults. El segundo es la generación de AdaptedContent en TeacherGeneratorView, que dispara una llamada asíncrona a AIAdaptationService y mantiene un estado isGenerating que la vista usa para mostrar un loader a pantalla completa. El tercero es el flag isApproved dentro de AdaptedContent, que solo cambia cuando el

docente lo confirma a mano y que es condición indispensable para que el siguiente paso, `publishClass()`, pueda ejecutarse. Así es como RN-02 se cumple de forma estricta.

Toda la sincronización entre vistas se hace con `@EnvironmentObject` y `@Published`. Los cuatro stores principales (`UserStore`, `ClassroomStore`, `SpeechService` y `NeurodiverseModeStore`) se inyectan una sola vez en `AdaptaMente1App` y bajan por todo el árbol de SwiftUI. Las vistas se suscriben de forma reactiva: cualquier cambio en una propiedad publicada vuelve a dibujar a quienes dependen de ella. No usamos `Combine` de forma explícita más allá de lo que SwiftUI ya integra.

5.1 Diagrama de estados resumido

Describimos el grafo de estados en forma de tabla. Preferimos esto a un diagrama de cajas porque nos deja anotar también los predicados de cada transición.

Estado origen	Evento	Estado destino
Splash	Timer de 2.0 s + fundido de 0.5 s	Router (ContentView)
ContentView	<code>hasCompletedOnboarding == false</code>	WelcomeView
ContentView	<code>hasCompletedOnboarding == true</code>	RoleSelectionView
WelcomeView	Toca 'Soy maestro'	RoleSelectionView (modo maestro, sin VAK)
WelcomeView	Toca 'Soy estudiante'	LearningStyleTestView
LearningStyleTestView	Termina el cuestionario	RoleSelectionView
RoleSelectionView	Toca maestro	TeacherGeneratorView
RoleSelectionView	Toca estudiante	StudentHubView
StudentHubView	Toca clase	JoinClassView
StudentHubView	Toca autodidacta	SelfStudyView
StudentHubView	Toca modos accesibles	NeurodiverseHubView
TeacherGeneratorView	Aprobar y publicar	Inserta PublishedClass y vuelve al Generator
NeurodiverseHubView	Toca dislexia	DyslexiaModeView
NeurodiverseHubView	Toca TDAH	TDAHFocusModeView
TDAHFocusModeView	5 tarjetas consumidas	BreathBreakOverlay (30 s, bloqueante)
BreathBreakOverlay	La cuenta llega a 0	Regresa a la tarjeta actual y reinicia el contador

6. Gestos e interacciones especiales

La app usa el repertorio de gestos habitual de iOS y le agrega un par de extensiones para los modos de neurodiversidad. Documentamos aquí los gestos que no son triviales: aquellos donde el comportamiento se aparta del que viene por defecto o donde metimos lógica háptica adicional.

Gesto	Pantalla	Comportamiento implementado
Tap	Todas las tarjetas de navegación	NavigationLink push estándar, con <code>Haptics.soft</code> al inicio del toque.
Tap para voltear	KinestheticAdaptationView (flashcards)	Anima una rotación 3D con <code>animation(.spring)</code> y dispara <code>Haptics.selection</code> al voltear.
Arrastre horizontal	TDAHFocusModeView	DragGesture con umbral de 80 puntos. Por debajo, la tarjeta vuelve a su sitio con <code>spring</code> . Por encima, sale con una rotación proporcional al desplazamiento y dispara <code>Haptics.medium</code> . Si era la última, pasa al estado <code>completed</code> .
Arrastre (emparejar)	KinestheticAdaptationView (modalidad emparejar)	DragGesture sobre tarjetas de concepto que se sueltan sobre las de definición. Valida en <code>onEnded</code> con <code>Haptics.success</code> o <code>Haptics.error</code> .
Deslizamiento horizontal	KinestheticAdaptationView (flashcards)	TabView paginado con interpolación elástica. Sin háptica adicional.
Pulsación larga	Ninguna pantalla del MVP	Reservada para una futura función de feedback contextual.
Pellizco	Ninguna pantalla del MVP	El mapa mental se dibuja con dimensiones fijas y no tiene zoom.

Gesto	Pantalla	Comportamiento implementado
Toggle on/off	DyslexiaModeView	Toggle nativo de SwiftUI con tinte #86E5B6. Cada cambio dispara save() en NeurodiverseModeStore (escritura en UserDefaults).

Tomamos una decisión a conciencia: los gestos que no son estándar siempre van acompañados de una pista de texto visible. En el modo Foco TDAH, por ejemplo, el pie de pantalla dice de forma explícita «← desliza para avanzar →» junto a un icono de una mano arrastrando. Lo consideramos necesario para la descubribilidad que pide la IA centrada en el humano, sobre todo en usuarios que quizá tengan menos contacto con apps recientes.

7. Datos, servicios y almacenamiento

Aquí detallamos los datos que maneja la app, los servicios externos a los que se conecta, qué operaciones hace contra cada uno y cómo guarda la información de forma local. Lo dividimos en tres partes. Primero el único servicio en la nube que consumimos, luego los servicios nativos del sistema operativo y, al final, un resumen por pantalla de las operaciones y de la persistencia.

7.1 Servicio externo: Google Gemini API

El único servicio en la nube que usamos es la API de Google Gemini, en concreto el modelo gemini-2.5-flash. Lo llamamos solo desde la capa de servicios del docente (AIAdaptationService.shared) para producir las tres adaptaciones VAK a partir del texto que entrega el maestro. El estudiante nunca toca la API: recibe únicamente el resultado ya curado y aprobado.

Atributo	Detalle
Endpoint base	https://generativelanguage.googleapis.com/v1beta/models/gemini-2.5-flash:generateContent
Protocolo	REST sobre HTTPS, con JSON de entrada y de salida
Autenticación	API key como parámetro de query string (?key=Alza...). Como alternativa, el header Authorization: Bearer.
Formato de petición	JSON con los campos contents (un arreglo de partes role/text) y generationConfig (temperature, maxOutputTokens y responseMimeType: application/json).
Formato de respuesta	JSON con candidates[].content.parts[].text, que parseamos a nuestras estructuras Codable AdaptedContent.
Cuota gratuita	60 peticiones por minuto y 1500 por día en el tier gratuito.
Costo de producción	Cerca de USD 0.075 por millón de tokens de entrada y USD 0.30 por millón de tokens de salida.
Estimación mensual MVP	En un escenario hipotético de 50 docentes activos generando 10 lecciones cada uno (≈ 1500 tokens por lección), el costo rondaría menos de USD 0.50 al mes.
Operaciones esperadas	Solo escritura, en un sentido, desde el cliente (POST). Gemini no guarda ni reutiliza nuestras peticiones.
Fallback	Si la API falla o no hay conexión, AIAdaptationService propaga el error y TextAdaptationEngine entra con el motor local NLP.

7.2 Servicios nativos del sistema operativo

Además del servicio externo, la app se apoya en cuatro frameworks nativos de iOS. Todos son gratuitos, no piden autenticación y corren por completo en el dispositivo. Eso no es casualidad: cumple la regla RN-03 sobre privacidad y nos permite seguir funcionando sin conexión.

Framework	Uso en la app	Costo	Autenticación
NaturalLanguage	Tokenización por oraciones, detección de idioma, embeddings para el clasificador VAK y para el motor local NLP que sustituye a Gemini cuando no hay conexión.	Cero	No
AVSpeechSynthesizer	Síntesis de voz para AudioAdaptationView y DyslexiaModeView, con voz nativa en español mexicano (com.apple.voice.compact.es-MX.Paulina) y respaldo en es-ES y es.	Cero	No
PDFKit	Lectura y extracción de texto de los PDF que el docente o el estudiante carga en TeacherGeneratorView y SelfStudyView.	Cero	No

Framework	Uso en la app	Costo	Autenticación
Core Haptics / UIKit haptics	Feedback táctil en tres intensidades (light, medium, heavy) y notificaciones (success, warning, error) en gestos críticos como los swipes del TDAH y la validación kinestésica.	Cero	No

7.3 Operaciones por pantalla y almacenamiento local

La tabla siguiente resume, pantalla por pantalla, las operaciones CRUD que se ejecutan y si hay escritura a almacenamiento persistente. Toda la persistencia va por UserDefaults, codificando estructuras Codable como datos JSON. Descartamos Core Data y SQLite a propósito, porque el volumen que manejamos (perfiles individuales, aulas locales, preferencias) cabe de sobra en UserDefaults, y la complejidad de Core Data no se justifica para el MVP.

Pantalla	Operaciones	¿Almacena localmente? Qué y por qué
SplashView	Ninguna	No
WelcomeView	Escritura: marca el rol elegido	Sí. UserProfile en UserDefaults, para conservar la decisión en aperturas siguientes.
LearningStyleTestView	Lee el banco de preguntas (JSON del bundle) y escribe el resultado	Sí. UserProfile.dominantStyle, scores y confidence, para no repetir el quiz cada vez.
RoleSelectionView	Lee UserProfile	No escribe, solo lee.
TeacherGeneratorView	Lee el PDF local, hace POST a Gemini y escribe PublishedClass	Sí. ClassroomStore.classes en UserDefaults, para que las aulas sigan disponibles sin conexión y entre sesiones.
StudentHubView	Lee UserProfile	No
JoinClassView	Lee ClassroomStore (búsqueda por código)	Solo lectura sobre un dato ya persistido.
SelfStudyView	Lee PDF/texto y llama a Gemini de forma opcional	No persiste. El material autodidacta es volátil por diseño, para no acumular basura.
VisualAdaptationView	Lee AdaptedContent	No
AudioAdaptationView	Lee AdaptedContent y controla AVSpeech	No
KinestheticAdaptationView	Lee AdaptedContent y rastrea el progreso	No guarda el progreso entre sesiones (decisión consciente que podría cambiarse más adelante).
NeurodiverseHubView	Ninguna	No
DyslexiaModeView	Lee y escribe preferencias al cambiar un toggle	Sí. NeurodiverseModeStore en UserDefaults, para que el usuario no reconfigure cada vez.
TDAHFocusModeView	Lee preferencias y sourceText	Sí, solo las preferencias (tarjetas por pausa y segundos de respiración).

8. Dispositivos, tamaños, orientaciones y sensores

8.1 Dispositivos soportados

La app está hecha para iPhone con iOS 17.0 o superior, solo en orientación vertical. No soportar horizontal responde a dos razones: las pantallas se diseñaron con una jerarquía vertical de lectura que pierde sentido al girar, y la rotación suma complejidad de layout que no le aporta nada al usuario del MVP. Tampoco soportamos iPad por ahora, aunque la base en SwiftUI permitiría adaptarla sin demasiado esfuerzo más adelante.

Modelo	Pantalla	Resolución lógica	Soporte
iPhone SE (3.ª gen)	4.7"	375 × 667	Sí
iPhone 13 / 14 / 15 / 16	6.1"	390 × 844	Sí
iPhone 14/15/16 Plus	6.7"	428 × 926	Sí
iPhone 15/16 Pro	6.1"	393 × 852	Sí
iPhone 15/16 Pro Max	6.7"	430 × 932	Sí
iPad (todos)	N/A	N/A	No en el MVP, quedaría para más adelante.
iPhone con iOS < 17	N/A	N/A	No (faltan funciones de SwiftUI requeridas).

8.2 Sensores y capacidades de hardware

La app solo usa el Taptic Engine de los iPhone para la háptica. No tocamos cámara, micrófono, giroscopio, acelerómetro, GPS, sensor de proximidad ni reconocimiento facial. La decisión es deliberada y nos deja un perfil de privacidad mínimo: ningún permiso sensible aparece al instalar y la app sirve incluso en modo avión, salvo por la generación con Gemini.

Sensor / capacidad	Uso en la app	Permiso requerido
Taptic Engine	Háptica en TDAHFocusModeView (swipes), KinestheticAdaptationView (validación de respuestas) y en confirmaciones críticas como publicar o aprobar.	Ninguno. UIImpactFeedbackGenerator no pide permiso.
Pantalla táctil	Toda la interacción es por toque. Aprovechamos gestos compuestos (arrastre + rotación) en el modo TDAH.	Ninguno
Acelerómetro / Giroscopio	Sin uso en el MVP. Pensamos en girar el dispositivo como gesto de descarte en TDAH para más adelante.	No aplica
Cámara	Sin uso. Descartada a propósito por privacidad (ver el capítulo 2, sobre la inviabilidad del OCR de pizarrones).	No lo pedimos
Micrófono	Sin uso en el MVP. Dejamos reservado NSMicrophoneUsageDescription en Info.plist para una futura función de dictado.	Reservado (sin pedirlo activamente)
GPS	Sin uso. La app no necesita ubicación.	No lo pedimos

9. Maqueta de pantallas finales

Para mostrar el resultado visual final del MVP recurrimos directamente a capturas de la aplicación ya implementada, con sus colores, datos de ejemplo y elementos terminados. A diferencia de los wireframes del capítulo 4, estas son pantallas reales tomadas del simulador y de dispositivos físicos. El diseño sigue las Human Interface Guidelines de Apple en paleta, tipografía, jerarquía y feedback de estado, y aplica los tokens del sistema de diseño que documentamos en Theme.swift. Las capturas siguen el recorrido completo, de la apertura de la app hasta la publicación de un aula.

9.1 Apertura y menú personalizado



Figura 9.1. Splash con el logo Brain-A



Figura 9.2. Menú principal personalizado

El Splash arranca con el logotipo Brain-A construido en SwiftUI nativo (Shapes y Path). El cerebro en azul zafiro lleva cinco nodos en verde menta suave que laten con una animación continua easeInOut con autoreverses. La marca aparece en SF Pro Rounded Bold con tracking ampliado y la tagline en gris suave. Los colores son los tokens HIG #0A66FF (zafiro) y #86E5B6 (menta) sobre fondo #F8F9FA. El menú aplica el mismo sistema: tarjeta de saludo con avatar, badge del estilo

dominante con su porcentaje de confianza, banner de privacidad con candado en verde menta, y dos tarjetas de rol con icono dentro de un cuadro de color y su disclosure indicator. Las sombras son muy suaves (radius 12, opacity 0.05, offset y=4).

9.2 Generación del docente y versión visual



Figura 9.3. Generador docente con las tres adaptaciones

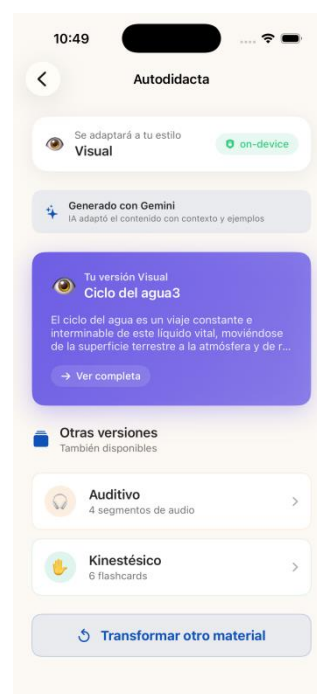


Figura 9.4. Versión visual con infografía y datos reales

En el generador, el docente ve las tres adaptaciones VAK en tarjetas paralelas (visual, auditiva y kinestésica) listas para revisar, editar y aprobar antes de publicarlas al aula. La versión visual del estudiante, sobre el ejemplo «El Viaje Intermitente del Agua», combina un resumen, un mapa mental y una infografía con datos puntuales (por ejemplo, los porcentajes de cada fase del ciclo) acompañados de iconos SF Symbols. Todo respeta la misma paleta y la jerarquía tipográfica del sistema de diseño.

9.3 Publicación de aula y modo Dislexia

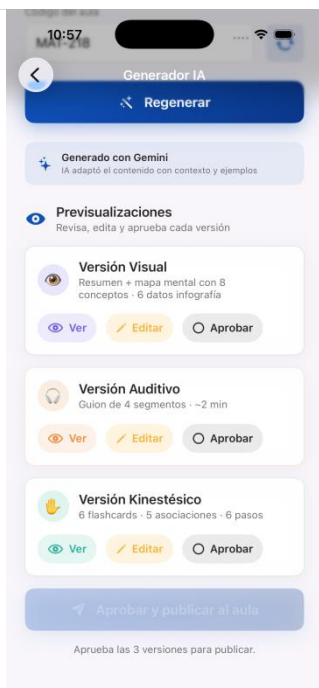


Figura 9.5. Aula publicada con su código



Figura 9.6. Modo Dislexia con fondo crema

Al aprobar el material, la app confirma la publicación y muestra el código de aula (en la captura, MAT-218) que el docente comparte con sus alumnos. El modo Dislexia trabaja por completo sobre fondo crema #FBF4E2 para quitar el halo retiniano del blanco puro. El texto va en una fuente legible con espaciado ampliado y, durante la lectura en voz alta, la palabra activa se resalta en ámbar #F4C95D en sincronía con AVSpeech. Estas tres a cinco pantallas resumen el flujo de extremo a extremo del producto con su apariencia final.

10. Lenguajes y herramientas de desarrollo

La app se escribe por completo en Swift 5.9 sobre el framework declarativo SwiftUI, con algún componente puntual de UIKit para la háptica y para configurar la sesión de audio. En el cliente no usamos ningún otro lenguaje ni runtime: toda la lógica vive en Swift compilado de forma nativa a binario iOS.

Como la app no necesita servidor propio (el único endpoint externo es Google Gemini), no tenemos backend. El consumo de Gemini se hace directo desde el cliente con URLSession y un decodificador Codable, sin proxy de por medio. Si en una versión futura sumáramos backend (por ejemplo, para analítica institucional o para esconder la API key detrás de un intermediario), nuestra primera opción sería Node.js con TypeScript por la familiaridad del equipo, o bien Vapor (Swift en el servidor) para quedarnos con un solo lenguaje.

10.1 Herramientas de desarrollo

Categoría	Herramienta	Versión
IDE principal	Xcode con Swift Package Manager	15.2+
Lenguaje cliente	Swift	5.9+
Framework UI	SwiftUI con UINavigationController	iOS 17 SDK
Frameworks auxiliares	UIKit (solo Haptics y AVAudioSession), Combine (implícito vía SwiftUI)	iOS 17 SDK
Control de versiones	Git + GitHub	N/A
Gestión de paquetes	Swift Package Manager (sin CocoaPods)	N/A
Generación de documentos	Node.js con la librería docx para los reportes y pptxgenjs para las presentaciones	Node 18+
Conversión a PDF	LibreOffice headless desde scripts auxiliares	7.x
IA externa	Google AI Studio (gestión de la API key)	N/A

11. Arquitectura y patrones de diseño

La app sigue una variante ligera de MVVM (Model-View-ViewModel) ajustada a cómo funciona SwiftUI. En vez de un ViewModel explícito por vista, usamos ObservableObject como capa de coordinación entre la vista y el modelo, que es lo idiomático en SwiftUI y recorta mucho boilerplate. Los stores (UserStore, ClassroomStore, SpeechService y NeurodiverseModeStore) hacen las veces de ViewModels compartidos y se inyectan en el árbol con @EnvironmentObject.

11.1 Capas de la arquitectura

Capa	Responsabilidad	Ejemplos en el código
App	Punto de entrada @main. Crea y posee los stores globales. Alterna entre el Splash y el router.	AdaptaMente1App.swift
Views (Presentación)	Vistas declarativas de SwiftUI. No llevan lógica de negocio, solo consumen el estado de los stores.	Carpeta Views/ (14 archivos)
Components	Subvistas reutilizables (PrimaryButton, SectionHeader, Chip, StyleBadge, etc.), sin estado propio.	Components/Components.swift
Theme	Sistema de diseño: tokens de color, tipografía, espaciado, radios y modifiers (Glassmorphism, CardStyle).	Theme/Theme.swift
Services (Lógica)	Servicios singleton ObservableObject que orquestan las APIs externas, la persistencia y el procesamiento.	Carpeta Services/ (10 archivos)
Models (Dominio)	Estructuras Codable que representan las entidades del dominio, sin lógica.	Models/Models.swift

11.2 Patrones de diseño aplicados

La arquitectura aplica cinco patrones que conviene documentar porque se ven tanto en el código como en el comportamiento del sistema:

Patrón	Implementación	Beneficio
Singleton	TextAdaptationEngine.shared, AIAdaptationService.shared y LearningStyleClassifier.shared.	Un punto único de acceso para servicios sin estado pesado, sin necesidad de inyección compleja para utilidades.
Strategy + Fallback	TextAdaptationEngine.adapt(...) decide en tiempo de ejecución entre la IA real y el motor local según AIConfig.isAIEnabled y la red disponible.	Continuidad del servicio sin conexión, sin cambiar las vistas que lo consumen.
Observer (vía Combine)	Stores marcados con @Published y vistas suscritas con @EnvironmentObject.	Sincronización reactiva automática. SwiftUI redibuja sin código imperativo.
Dependency Injection	Los stores se inyectan desde AdaptaMente1App con .environmentObject() y las vistas los declaran con @EnvironmentObject.	Las pruebas pueden sustituir stores sin tocar las vistas, y hay un solo lugar para configurar dependencias.
Máquina de estados implícita	Variables @State y @Published modelan transiciones discretas (showSplash, hasCompletedOnboarding, isApproved, showBreathBreak).	Comportamiento predecible y fácil de rastrear.

12. Metodología de desarrollo y administración del proyecto

Adoptamos una versión ligera de Kanban como metodología principal, sin las ceremonias completas de Scrum, porque el equipo es chico y el plazo del hackathon, corto. Llevamos un tablero en Notion con cinco columnas (Backlog, En análisis, En desarrollo, En revisión y Hecho) y revisábamos el avance en reuniones cortas.

Cada tarjeta es un requerimiento funcional o una tarea técnica que se desprende de él. Llevan etiqueta de tipo (feature, bug, refactor, documentación), prioridad (P0 bloqueante, P1 importante, P2 deseable) y responsable. Cuando una tarjeta pasa a En desarrollo, su responsable crea una rama Git con el nombre del ticket, trabaja ahí y abre un Pull Request al cerrar. Toda PR la revisa al menos otro integrante antes de mergear a main.

12.1 Herramientas de administración

Herramienta	Uso
Notion	Tablero Kanban, documentación viva, actas de reunión y lluvia de ideas.
GitHub	Versionado, Pull Requests y seguimiento de issues técnicos.
WhatsApp	Comunicación asíncrona rápida para decisiones de poca gravedad.
Google Meet	Reuniones de planeación y demos internas del equipo.
Xcode Cloud (opcional, a futuro)	Pipeline de build en la nube de Apple y generación de TestFlight.

12.2 Sprints y entregas internas

El grueso del desarrollo se concentró en los dos días del hackathon. Antes del evento dedicamos algunas sesiones a investigar el tema, definir el enfoque y preparar los wireframes y el sistema de diseño. Durante el primer día levantamos el onboarding, el cuestionario VAK y las tres vistas de consumo. Durante el segundo cerramos el modo maestro, los modos accesibles y la integración con Gemini, y dejamos tiempo para probar en dispositivo y pulir detalles. La redacción de este documento la hicimos después, ya fuera del evento.

13. Uso de IA durante nuestro desarrollo

Hay que separar la IA que la app usa como producto de la IA que nosotros usamos como herramienta mientras la construíamos. La primera es Gemini 2.5 Flash y ya quedó documentada en el capítulo 7. La segunda es de la que hablamos aquí: el uso instrumental de modelos generativos para acelerar nuestro propio trabajo de ingeniería.

Usamos tres herramientas de IA durante el desarrollo. La primera es Claude (Anthropic), para pair programming de arquitectura, redacción de documentación técnica y para sostener este mismo trabajo escrito. Trabajamos con prompts pensados que le asignan un rol experto (por ejemplo, «actúa como Tech Lead iOS y refactoriza esta vista para que respete las HIG»). La segunda es GitHub Copilot integrado a Xcode, que aceleró el boilerplate repetitivo (constructores Codable, extensiones de Color, etc.). La tercera es Gemini (el mismo servicio que la app consume en producción), que usamos como segunda opinión para revisar fragmentos de Swift menos comunes.

Fijamos tres principios para usar IA en el desarrollo. El primero, todo código asistido por IA pasa por revisión humana en Pull Request antes de mergear, con el mismo criterio que les pedimos a los docentes en RN-02. El segundo, la IA acelera pero no sustituye el pensamiento de arquitectura, así que las decisiones de diseño las escribimos y discutimos nosotros, en persona. El tercero, nunca compartimos con la IA datos sensibles de usuarios reales, y los ejemplos de los prompts son siempre sintéticos.

Herramienta	Uso concreto	Validación humana
Claude	Generación del delegate de SpeechService, redacción del marco teórico y refactor de Theme.swift hacia el sistema HIG.	Revisión en PR y lectura crítica antes de mergear.
GitHub Copilot	Autocompletado de boilerplate Codable, extensiones de Color y helpers de animación.	Lectura inline al aceptar cada sugerencia.
Gemini	Segunda opinión sobre fragmentos de Swift y validación de prompts antes de embeberlos en AIConfig.	Cotejo cruzado con la documentación oficial.

14. Permisos y consideraciones para publicación en App Store

Para publicar en la App Store tenemos que cumplir tres conjuntos de requisitos: el formal del programa de desarrolladores, el de las App Store Review Guidelines y el de privacidad, que Apple endureció en iOS 17.

14.1 Programa Apple Developer

La cuenta necesaria es la Apple Developer Program de paga, con un costo de USD 99 al año. Permite firmar binarios, distribuir por TestFlight a beta testers y publicar en la tienda. Como somos un equipo estudiantil, podemos aplicar al Apple Developer Program for Educators, que es gratuito pero limita la distribución a cien dispositivos y no permite publicar en la tienda. Si en algún momento quisiéramos publicarla de verdad en la tienda, habría que pagar la cuenta comercial.

14.2 App Store Review Guidelines relevantes

Sección	Requisito	Cómo lo cumplimos
1.3 Apps dirigidas a menores	Si el público incluye menores, exige una clasificación específica y prohíbe ciertos comportamientos de tracking.	Nuestro MVP tiene rating 4+. No usamos tracking ni publicidad.
2.1 App Completeness	El binario subido debe ser estable y completo, no un placeholder.	El MVP cumple los cinco criterios de aceptación del punto 3.3.
5.1 Privacy	Los datos personales recolectados deben tener un manifiesto claro y el usuario debe poder borrarlos.	No recolectamos datos. El perfil VAK es local y se borra con el botón 'Rehacer diagnóstico'.
5.1.1 Data Collection	El Privacy Manifest (PrivacyInfo.xcprivacy) es obligatorio desde iOS 17.	Lo incluimos declarando 'no data collected'.
5.2 Intellectual Property	El contenido generado por IA debe respetar derechos.	El usuario aporta el material fuente y el motor solo lo reorganiza.

14.3 Privacy Manifest y App Store Connect

Desde iOS 17, toda app debe incluir en el bundle un archivo PrivacyInfo.xcprivacy que declare las APIs sensibles que usa (UserDefaults entra en la lista) y las categorías de datos recolectados. En nuestro caso declaramos NSPrivacyAccessedAPICategoryUserDefaults con la razón CA92.1 (acceso a nuestras propias defaults) y dejamos vacío el arreglo de datos recolectados, porque de verdad no recolectamos ninguno. Las Privacy Nutrition Labels en App Store Connect mostrarían 'Data Not Collected', algo poco habitual en apps educativas.

15. Equipo de trabajo y roles

El equipo lo formamos tres integrantes, todos estudiantes de Ingeniería en Computación o carreras afines en la Facultad de Ingeniería de la UNAM. Repartimos los roles según las fortalezas de cada quien, y los mantuvimos flexibles: cualquiera podía tomar tareas de otro cuando la carga lo pedía.

Integrante	Rol primario	Responsabilidades
Caballero Montaña Juan Manuel	Tech Lead iOS	Arquitectura general, decisiones de stack, integración de servicios y revisión final de los Pull Requests.
Camarena Arévalo Yael Eduardo	Ingeniero de IA y Backend	Integración con Gemini, motor local NLP, diseño de prompts, manejo de errores de red y estrategia de fallback.
Cruz Soria Oscar	Diseño UI/UX, SwiftUI dev y Product/QA	Sistema de diseño (Theme.swift), wireframes y maquetas, implementación de vistas y glassmorphism, tablero Kanban, pruebas en dispositivos físicos, documentación y coordinación con el docente.

Los tres aportamos a este trabajo escrito en proporciones parecidas. Juan Manuel llevó los capítulos técnicos 10 y 11, Yael los capítulos 7 y 13 y la parte de IA del 2, y Oscar los capítulos 4, 9 y 12. Los capítulos restantes los trabajamos en conjunto.

16. Estimaciones de tiempo y costo

Presentamos dos estimaciones: la real, con las horas que de verdad le metimos al MVP, y la proyectada, que extrapola el costo si lleváramos el producto a publicación comercial con un equipo contratado a tarifas del mercado mexicano.

16.1 Tiempo invertido en el MVP (real)

Actividad	Detalle	Horas (equipo)
Investigación y marco teórico	Lectura de fuentes, definición del enfoque y referencias	9
Wireframes y sistema de diseño	Bocetos de pantallas, Theme.swift y maquetas	9

Actividad	Detalle	Horas (equipo)
Hackathon día 1	Onboarding, cuestionario VAK y las tres vistas de consumo	12
Hackathon día 2	Modo maestro, modos accesibles e integración con Gemini	12
Pruebas y pulido	Pruebas en iPhone 13 y 15 Pro, ajustes de HIG y Splash	6
Redacción del trabajo	Documento escrito y presentación	9
Total	Proyecto completo	57 horas

El grueso del desarrollo ocurrió en los dos días del hackathon, donde cada integrante le dedicó alrededor de 6 horas por día. El resto (investigación, wireframes, pruebas y la redacción de este documento) sumó unas 7 horas más por persona. En total son cerca de 19 horas por persona (unas 57 horas de equipo). Fue un esfuerzo intenso y acotado, propio de un proyecto escolar, no de un desarrollo repartido a lo largo de muchas semanas.

16.2 Estimación proyectada a producción comercial

Si en lugar de un equipo estudiantil contratáramos a uno profesional para llevar el MVP hasta publicarlo con calidad comercial, las cifras serían las siguientes, calculadas con tarifas medias del mercado mexicano de desarrollo de software en 2026:

Concepto	Detalle	Costo MXN
Apple Developer Program	Membresía anual obligatoria para publicar	\$1,800
Dominio y correo institucional	.org y 4 correos profesionales al año	\$1,200
Tarifa Gemini año 1	50 docentes × 10 lecciones/mes, con holgura	\$1,500
Iconografía y branding final	Logo, splash, asset catalog y mockups para la App Store	\$8,000
Auditoría de accesibilidad	Especialista externo (VoiceOver, Dynamic Type)	\$6,000
Pruebas en device farm	60 horas en una flotilla de dispositivos reales (BrowserStack o similar)	\$4,500
Servicios legales	Revisión de T&C y política de privacidad por abogado, mercado mexicano	\$5,000
Marketing inicial (3 meses)	App Store Optimization y redes sociales	\$12,000
Subtotal inicial (lanzamiento)		\$40,000
Mantenimiento mensual proyectado	1 dev iOS senior part-time, soporte a usuarios y costo de Gemini a escala	\$15,000 / mes

Esa proyección parte de un escenario conservador. Si el producto creciera por encima de mil aulas activas, los costos de Gemini subirían de forma lineal, algo que ya contemplamos en la estrategia freemium del capítulo de reglas de negocio: la generación con Gemini es exclusiva del tier de paga, mientras que el gratuito usa el motor local NLP, lo que mantiene el costo variable bajo control.

17. Referencias

Las referencias siguientes sustentan tanto la base teórica de la aplicación como las decisiones técnicas que tomamos. Están en formato APA y se citan a lo largo del documento por autor y año.

Apple Inc. (2024). Human Interface Guidelines: iOS. Apple Developer Documentation. <https://developer.apple.com/design/human-interface-guidelines/>

Apple Inc. (2024). Privacy Manifest Files. Apple Developer Documentation. https://developer.apple.com/documentation/bundleresources/privacy_manifest_files

Apple Inc. (2024). AVSpeechSynthesizer. Apple Developer Documentation. <https://developer.apple.com/documentation/avfaudio/avspeechsynthesizer>

Apple Inc. (2024). Natural Language Framework. Apple Developer Documentation. <https://developer.apple.com/documentation/naturallanguage/>

Fundación UNAM. (2019). El TDAH en México: estadísticas y desafíos. <https://www.fundacionunam.org.mx/>

Google. (2024). Gemini API Documentation. Google AI for Developers. <https://ai.google.dev/gemini-api/docs>

- IMARC Group. (2024). Mexico Educational Apps Market Report 2024-2033. <https://www.imarcgroup.com/>
- Mexicanos Primero. (2024). Estado de la Educación en México 2024. <https://www.mexicanosprimero.org/>
- Red por los Derechos de la Infancia en México (REDIM). (2025). Informe sobre deserción escolar. <https://derechosinfancia.org.mx/>
- Rello, L., & Baeza-Yates, R. (2013). Good fonts for dyslexia. Proceedings of the 15th International ACM SIGACCESS Conference on Computers and Accessibility (ASSETS '13). <https://doi.org/10.1145/2513383.2513447>
- Shneiderman, B. (2020). Human-Centered Artificial Intelligence: Reliable, Safe & Trustworthy. International Journal of Human-Computer Interaction, 36(6), 495-504. <https://doi.org/10.1080/10447318.2020.1741118>